VR-Audit

A Programme of Operational Audits of Classical Theorems

Vitaly Reznik

Version 1.0.1 — 1 June 2026

Abstract

VR-Audit is an open-ended programme of applying the two-register apparatus of VR-Forms to classical theorems of mathematics. Each audit establishes an operational corollary of a classical theorem through the transit pattern: classical machinery is used as a black box in the formal register, while operationality of the result is established separately through wrapping predicates and explicit witness construction.

This preprint accumulates the work of the programme. Version 1.0.0 contains the first audit: the Hahn–Banach extension theorem for operational Hilbert spaces, proved via Riesz representation. Future audits will be added as new versions, with each version superseding the previous as a complete record of the programme to date.

The accompanying Lean 4 formalisation is published separately at Zenodo: DOI 10.5281/zenodo.20363739.

* * *

Part I — Position

I.1 Place in the VR Cycle

VR-Audit is the fifth work in the VR Cycle, following VR (A Formal System), VR-Numbers, VR-Sets, and VR-Forms. The first four works are foundational: they establish an operational foundation rooted in the empty set, with two registers (formal and operational) connected by a transit principle.

VR-Audit is applied. It demonstrates the apparatus on classical theorems with mathematical content, using mathlib and the predecessor Lean cycles as black-box dependencies.

I.2 What an audit is

An audit consists of:

- Choice of a classical theorem from mathlib or established mathematical practice.
- Identification of the operational restrictions on the inputs sufficient to produce an operational output via transit.

- Definition of operational types as predicate restrictions over mathlib's classical types (wrapping principle).
- Proof in Lean 4 that, given operational inputs, the classical theorem yields an output equipped with operational witnesses.
- Verification that the Specker boundary does not bite for the specific transit structure.

The audit does not re-prove the classical theorem. It establishes an operational corollary on operational inputs.

I.3 What this programme is not

VR-Audit is not a competitor to Bishop-style constructive analysis. Bishop's programme requires every step of every proof to be constructive, philosophically refusing classical machinery. VR-Audit admits classical machinery in the formal register and produces operational results via transit. The trade-off is between philosophical purity (Bishop) and engineering efficiency (VR-Audit).

VR-Audit is not a foundational programme in itself. It rests on VR-Sets and VR-Forms for its foundational apparatus. Its contribution is demonstrating that the apparatus works on classical mathematics with realistic complexity.

VR-Audit is not a classification programme like reverse mathematics. Reverse mathematics classifies theorems by their axiom strength over RCA_0. VR-Audit reorganises theorems by their operational versus formal content under a fixed apparatus. The two approaches are complementary.

* * *

Part II — Architecture

II.1 The wrapping principle

VR-Audit does not introduce parallel types for operational objects. There is no OperationalReal distinct from mathlib's Real. Instead, predicates select operational sub-collections from mathlib's classical types.

The basic predicate is:

```
IsComputableReal (x : ℝ) : Prop :=
  ∃ (alg : ℕ → ℚ) (mod : ℕ → ℕ),
    ∀ n k, mod n ≤ k → |alg k - x| ≤ 1 / 2^n
```

A classical real is computable if there exists a rational approximating sequence with an explicit modulus of convergence. The predicate is Prop, not Type; the witness data is recovered via existential elimination when needed.

The same pattern extends to Hilbert spaces, subspaces, and functionals: each is mathlib's classical structure equipped with additional fields asserting computability of finite witness data.

II.2 Two-register correspondence

In VR-Forms language: the formal register contains all of mathlib's classical types and theorems, with full use of Classical.choice. The operational register contains the sub-collection of objects satisfying the wrapping predicates, with explicit witnesses.

A transit theorem takes operational inputs (carrying witnesses), invokes classical machinery from mathlib, and produces operational outputs (with witnesses constructed from the inputs). The classical machinery in the middle does not affect the operational status of the result, provided witness construction at the boundaries is explicit.

II.3 Lean's algorithmic content

The wrapping predicates do not use Lean's Computable typeclass. The reason is structural: Computable_2 for rational arithmetic operations (+, *, − on $\mathbb{Q}$) is absent from mathlib4 (verified by exhaustive search). Using Computable annotations on the witness functions would require building this missing infrastructure from scratch.

Instead, the witnesses use Lean's intrinsic totality of definable functions. Algorithmicity is provided at the type level by Lean's metatheory; stronger forms (Turing machine codings) remain metatheoretic and are not expressible as Lean predicates. This is consistent with the position documented in VR-Numbers Reals.lean: Lean 4 does not distinguish computable from non-computable functions at the type level. The operational stance restricts functions to those with finite algorithmic descriptions; this restriction is a metatheoretic claim not expressible as a Lean type predicate.

II.4 Acceptance of Classical.choice

Every public object in the VR-Audit Lean cycle has axiom profile [propext, Classical.choice, Quot.sound] — the standard mathlib ceiling. The use of Classical.choice is expected: mathlib's Hahn–Banach uses Zorn's lemma, and mathlib's Riesz representation uses the axiom of choice through dual space machinery. This is principled, not a defect. The methodological point of transit is precisely to use classical machinery while preserving operationality at the boundaries.

The operational content of an audit is not measured by axiom minimisation. It is measured by the explicit construction of output witnesses from input witnesses.

* * *

Part III — Audit One: Hahn–Banach for Operational Hilbert Spaces

III.1 The theorem

Let E be an operational Hilbert space (a classical Hilbert space equipped with a computable dense sequence and computable inner products on that sequence). Let M be an operational located subspace of E (a closed subspace for which distance from points of the ambient dense sequence is computable). Let f be an operational normable linear functional on M (a continuous linear functional with computable values on M's dense sequence and computable operator norm).

Then there exists a continuous linear functional g on E such that:

- g restricts to f on M;
- the operator norm of g equals the operator norm of f;
- g is operational: its values on E's dense sequence are computable, and its operator norm is computable.

III.2 Locatedness and normability

Two operational restrictions deserve emphasis.

Locatedness (Bishop): the distance from a point to the subspace is computable. Classically, every closed subspace is "located" in the trivial sense (the infimum exists), but constructively, computability of the infimum is not automatic. Locatedness is the constructive replacement for classical closedness.

Normability (Ishihara 1989): the exact operator norm exists as a computable real. In classical functional analysis, boundedness (existence of some constant C with $|f(x)| \leq C \cdot \|x\|$) is equivalent to the existence of the operator norm. Constructively the implications split: a bounded functional need not be normable. The VR-Audit formalisation tracks the distinction by selecting the normable sub-collection via the norm_computable predicate.

III.3 Proof structure

The proof proceeds in four steps.

Step 1 — Riesz representation. Mathlib's InnerProductSpace.toDual provides a linear isometry between the Hilbert space M and its continuous dual. Inverting this isometry on f yields a vector $\xi \in M$ such that $f(x) = \langle x, \xi \rangle$ for all $x \in M$. The inversion uses Classical.choice through the dual space construction. The vector ξ is obtained as a formal object — it "exists" in the sense of mathlib's classical machinery.

Step 2 — Definition of the extension. Define g on all of E by $g(x) = \langle x, \xi \rangle$. This is the inner-product operator with ξ, a continuous linear functional by general properties of the inner product. No classical machinery is invoked in this step.

Step 3 — Operationality of the extension. For g to be operational, the values g(denseSeq n) must be computable for every n. Direct verification is not possible: ξ is a formal object, and ⟨denseSeq n, ξ⟩ is just a classical real number a priori.

The operational extraction uses the orthogonal projection identity. The orthogonal projection P_M(denseSeq n) lies in M. By the identity ⟨x, ξ⟩ = ⟨P_M(x), ξ⟩ for x ∈ E and ξ ∈ M (a standard mathlib lemma), one has g(denseSeq n) = ⟨P_M(denseSeq n), ξ⟩, which by Riesz equals f(P_M(denseSeq n)).

Now f is operational on M's dense sequence by hypothesis, and an auxiliary lemma (Stage 4 of the Lean cycle) extends this to operationality on all of M via continuity. The projection P_M(denseSeq n) ∈ M, so f applied to it is computable. Therefore g(denseSeq n) is computable.

Step 4 — Norm equality. Mathlib's Riesz isometry gives ‖f‖ = ‖ξ‖_M. The norm of the inner-product operator is ‖g‖ = ‖ξ‖_E. The subspace coercion gives ‖ξ‖_M = ‖ξ‖_E. Therefore ‖g‖ = ‖f‖. Since ‖f‖ is computable by hypothesis, ‖g‖ is computable.

III.4 Where the apparatus does work

Two places in the proof are where VR-Forms apparatus does the essential work.

At Step 1: Bishop-style constructive mathematics would prohibit invoking classical Riesz. The constructive Riesz of Bishop-Bridges requires uniform convexity and explicit constructions over several pages. VR-Forms permits invocation of classical Riesz as a formal-register tool; the formal status of ξ is acknowledged and managed.

At Step 3: The operationality of g is not inherited from the classical machinery. It is constructed explicitly using locatedness of M, orthogonal projection, and operationality of f. This is the transit pattern in operation: the formal object ξ is used as an intermediate, but operational witnesses are built around it via structural arguments.

III.5 The Specker boundary

Specker (1949) constructed a monotone bounded computable sequence of rationals whose limit is not computable. The result generalises to any constructive setting where bounded sequences may have non-computable suprema. Many classical theorems whose conclusions involve suprema, infima, or other limit-like constructions cannot be transited directly into operational form: the conclusion may be a classical object without operational extraction.

In Hahn–Banach for Hilbert spaces via Riesz, the Specker boundary is avoided by structure. The Riesz representation produces a specific vector ξ, not a supremum of approximating extensions. The orthogonal projection produces a specific element of the subspace, not a limit of bounded sequences. At each step the apparatus stays away from the constructions that would invoke the Specker obstacle.

For other classical theorems where suprema or infima are essential (e.g., open mapping theorem in its general form), the audit will require different transit structures. Whether such transits exist within the VR-Forms apparatus is an open question for each candidate theorem.

III.6 Reference Lean object

The main theorem in the Lean cycle is HahnBanachOperational_Hilbert. Public objects total 17 across 6 files in VRCycle/Audit/. Total Lean code is approximately 1427 lines. All public objects carry axiom profile [propext, Classical.choice, Quot.sound]. No sorry, no admit.

The first non-trivial example, demonstrating the typeclass is non-vacuous, is instOperationalHilbertSpaceEuclidean: for every n, the Euclidean space $\mathbb{R}^n$ (constructed in mathlib as EuclideanSpace $\mathbb{R}$ (Fin n)) is an operational Hilbert space. The dense sequence is the enumeration of rational tuples through mathlib's Encodable infrastructure; the inner product on rational tuples reduces to a finite sum of rational products, which is rational and hence computable.

III.7 Cost comparison

Mathlib's classical proof of Real.exists_extension_norm_eq together with its immediate dependencies (the Hahn–Banach cone extension via Zorn's lemma, sublinear function infrastructure, normed space basics) is approximately 2000–4000 lines of Lean. The full transitive closure including general normed space and operator norm infrastructure is in the tens of thousands of lines, but most of this is reusable across many theorems and should not be attributed solely to Hahn–Banach.

A Bishop-style constructive proof of Hahn–Banach for Hilbert spaces, ported to Lean from existing constructive analysis literature (Bishop-Bridges 1985, Ishihara 1989), would require rebuilding operational reals, operational Hilbert space infrastructure, constructive Riesz representation with uniform convexity arguments, and constructive density arguments. Realistic estimate: 3000–5000 lines of Lean.

The VR-Audit proof of operational Hahn–Banach is approximately 150 lines, embedded in a total cycle infrastructure of approximately 600 lines. The efficiency multiplier is on the order of 5–8× relative to a Bishop-style constructive rewrite, and 15–25× relative to a direct rewrite of the classical proof from scratch in operational form.

The multiplier does not measure new mathematical content. It measures the reorganisation enabled by the two-register apparatus: existing classical infrastructure becomes usable for constructive purposes via transit, eliminating the need to reproduce that infrastructure.

* * *

Part IV — Methodological Observations

The following observations were accumulated during the work on Audit One. They inform the design of subsequent audits and are recorded here as the operational findings of the programme.

Observation 1 (algorithmic content as Lean totality). The wrapping predicate IsComputableReal uses unmarked total functions alg : $\mathbb{N} \to \mathbb{Q}$ and mod : $\mathbb{N} \to \mathbb{N}$, not Lean's Computable typeclass. The reason is the absence of Computable_2 for rational arithmetic in mathlib4. Lean's intrinsic totality of definable functions provides algorithmicity at the type level; stronger forms (Turing machine codings) remain metatheoretic. This is consistent with VR-Numbers Reals.lean.

Observation 2 (wrapping at the Hilbert space level). The field inner_computable : ∀ m n, IsComputableReal (⟨denseSeq m, denseSeq n⟩) is the embodiment of the wrapping principle on a structured mathematical object. No new "computable inner product" type is defined; computability is asserted of selected outputs of the classical inner product. The operational register lives inside the formal register through the predicate.

Observation 3 (operational predicates over operational witnesses). Locatedness is formulated over the ambient dense sequence (points carrying operational witnesses), not over arbitrary points of the Hilbert space. This is principled. An arbitrary point of E has no operational tag and lives entirely in the formal register; demanding its operational properties would conflate the registers. The pattern is general: operational predicates apply only to objects carrying explicit witnesses.

Observation 4 (Specker boundary by structure). The orthogonal projection produces a single element, not a supremum of approximations. The Riesz representation produces a single vector, not a limit of bounded sequences. At no point in the Hahn–Banach transit does the proof invoke a bounded monotone sequence whose limit might be non-computable. The Specker obstacle is avoided not by additional argument but by the structural choice of Riesz over Zorn-based extension.

Observation 5 (bounded vs normable after Ishihara). Classical equivalence of boundedness and existence of the operator norm fails constructively. Mathlib's ContinuousLinearMap formalises boundedness; VR-Audit's OperationalNormableFunctional adds normability through norm_computable. The distinction tracks Ishihara (1989) and is essential constructively.

Observation 6 (transit pattern for Hilbert HB). The proof structure is: (1) obtain a classical witness ξ via mathlib's Riesz; (2) define the extension explicitly via inner product with ξ; (3) extract operationality of the extension via orthogonal projection identity and fn_computable_everywhere; (4) inherit norm equality from Riesz isometry. Steps 1 and 4 use classical machinery; steps 2 and 3 are operational construction. This is the schematic pattern: classical existence → explicit definition → operational extraction via structural identity → operational norm via classical isometry.

Observation 7 (Norm synthesis gap in mathlib). Typeclass synthesis of Norm (↥M.toSubmodule →L[ℝ] ℝ) for submodule-subtype continuous linear maps does not succeed through mathlib's instance chain at declaration sites and goal positions, while direct field access f.opNorm : ℝ works. Wrapping predicates in operational definitions over such functionals must use opNorm rather than ‖·‖ notation, with the bridge ContinuousLinearMap.norm_def invoked explicitly at points where the two forms must be connected.

Observation 8 (@-form for cross-form instance synthesis). When a reducible structure accessor causes instance synthesis to search under a different syntactic form than a declared local instance (as happens with OperationalLocatedSubspace.toSubmodule and its underlying ClosedSubmodule), the engineering fix is to pass the instance explicitly via @, not to provide multiple aliases. The @-form is self-documenting and avoids fragile redirection. This is a portable Lean 4 engineering principle.

Observation 9 (apparatus efficiency). Audit One demonstrates that the two-register apparatus enables operational extension of a classical theorem at approximately 150 lines of Lean, contrasting with several thousand lines required for either a direct constructive proof in Bishop's style or a rewrite of the classical machinery from scratch in operational form. The multiplier is in the 5–25× range depending on the comparison baseline. The apparatus does not introduce new mathematical content; it reorganises the relationship between classical and constructive content.

Observation 10 (non-vacuity demonstration). The EuclideanSpace ℝ (Fin n) instance shows the OperationalHilbertSpace typeclass is non-vacuous on a non-trivial infinite family of structures. The construction uses mathlib infrastructure entirely (Encodable enumeration of rational tuples, PiLp.homeomorph between EuclideanSpace and Π i, ℝ); the operational fields reduce to finite rational sums, which are computable by IsComputableReal_rat from Stage 1. This is the wrapping principle's terminal demonstration: a non-trivial classical structure becomes operational through three additional predicates with no redefinition of underlying content.

* * *

Part V — Programme Outlook

V.1 The accumulating preprint

This document is intended as an accumulating record of the VR-Audit programme. Subsequent audits will be added as new parts within future versions of this preprint. Each version will supersede the previous as the complete record to date. This avoids fragmentation of the programme across many small preprints while keeping each individual audit individually citable through its part-numbered section and its corresponding Lean tag.

V.2 Candidate audits for future versions

The following classical theorems are candidates for subsequent audits, in approximate order of structural compatibility with the transit pattern as currently understood:

Banach–Steinhaus (uniform boundedness principle) for operational Banach spaces. Mathlib formalisation exists. Transit structure: pointwise boundedness on a dense sequence transports to uniform boundedness via Baire category, with operational extraction via the same dense witness machinery.

Open mapping theorem for operational Banach spaces. Mathlib formalisation exists. Transit structure: the open mapping property may interact with the Specker boundary through Baire category arguments; locatedness conditions on the image may be required.

Closed graph theorem for operational Banach spaces. Follows from open mapping; transit inherits.

Spectral theorem for self-adjoint compact operators on operational Hilbert spaces. Mathlib formalisation exists. Transit structure: eigenvalues form a sequence with explicit construction; operational extraction through the projection-valued spectral decomposition.

Stone–Weierstrass theorem for operational continuous function spaces. Mathlib formalisation exists. Transit structure: density witnesses translate directly.

The selection of the next audit is open. Each candidate raises specific questions about whether the transit structure is clean (as for Hilbert HB via Riesz) or whether the Specker boundary bites (potentially the case for open mapping in its full generality).

V.3 Open theoretical questions

Two questions raised by Audit One remain open:

Status of WKL_0 in VR-Sets. Brown–Simpson (1986) showed that separable Hahn–Banach is equivalent to Weak König's Lemma over RCA_0. Whether VR-Sets proves WKL_0 — and consequently whether separable Hahn–Banach (without the Hilbert specialisation) is operationally accessible — is not addressed by Audit One. This is a foundational question independent of the audit; it can be pursued separately within the VR-Sets framework.

Generalisation beyond Hilbert. Audit One restricts to Hilbert spaces because Riesz representation provides a structural mechanism that avoids the Specker boundary. For general Banach spaces, transit through classical Hahn–Banach (which uses Zorn) does not obviously yield operational extensions: the classical extension is a Zorn-maximal element with no constructive content. Whether some weaker operational result is achievable via different transit structures (e.g., for uniformly convex Banach spaces, following Ishihara 1989) is open.

V.4 Position relative to neighbouring programmes

VR-Audit operates in a landscape that includes Bishop-style constructive analysis (Bishop-Bridges 1985), constructive Hahn–Banach by Ishihara (1989), reverse mathematics (Simpson 1999), computable analysis (Pour-El–Richards 1989; Weihrauch 2000), Weihrauch reducibility analysis of Hahn–Banach (Gherardi–Marcone 2009), and machine-verified mathematics in proof assistants generally.

The contribution of VR-Audit is not new mathematics. The theorems audited are classical or constructively known. The contribution is methodological: a working demonstration that the two-register apparatus of VR-Forms enables operational corollaries of classical theorems with substantially lower engineering cost than constructive rewriting, and with explicit machine-verified evidence rather than philosophical argument.

The programme is open-ended in scope but bounded in claim: it does not assert that operational corollaries are available for all classical theorems, only that they are available for those theorems whose transit structures avoid the Specker boundary and whose inputs admit operational witnesses.

* * *

References

Aczel, P. (1988). *Non-Well-Founded Sets.* CSLI Publications.

Bishop, E. (1967). *Foundations of Constructive Analysis.* McGraw-Hill.

Bishop, E. & Bridges, D. (1985). *Constructive Analysis.* Springer.

Brown, D. K. & Simpson, S. G. (1986). Which set existence axioms are needed to prove the separable Hahn–Banach theorem? *Annals of Pure and Applied Logic*, 31, 123–144.

Gherardi, G. & Marcone, A. (2009). How incomputable is the separable Hahn–Banach theorem? *Notre Dame Journal of Formal Logic*, 50(4), 393–425.

Ishihara, H. (1989). On the constructive Hahn–Banach theorem. *Bulletin of the London Mathematical Society*, 21, 79–81.

Pour-El, M. B. & Richards, J. I. (1989). *Computability in Analysis and Physics.* Springer.

Reznik, V. (2026a). VR. A formal system. *Zenodo.* DOI: 10.5281/zenodo.20324391.

Reznik, V. (2026b). VR-Numbers. *Zenodo.* DOI: 10.5281/zenodo.20352239.

Reznik, V. (2026c). VR-Sets. *Zenodo.* DOI: 10.5281/zenodo.20354628.

Reznik, V. (2026d). VR-Forms. *Zenodo.* DOI: 10.5281/zenodo.20355939.

Reznik, V. (2026e). VR-Audit Lean 4 formalisation, v1.4-vr-audit-hb-hilbert. *Zenodo.* DOI: 10.5281/zenodo.20363739.

Simpson, S. G. (1999). *Subsystems of Second Order Arithmetic.* Springer.

Specker, E. (1949). Nicht konstruktiv beweisbare Sätze der Analysis. *Journal of Symbolic Logic*, 14(3), 145–158.

Weihrauch, K. (2000). *Computable Analysis: An Introduction.* Springer.

* * *

Acknowledgements

The work was developed using Claude Opus 4.7 (architectural review) and Claude Sonnet 4.6 (Lean implementation), Variant A (interactive parent-child architecture), consistent with all preceding Lean cycles of the VR Cycle. Version 1.0.1 reframes the exposition to the cycle's no-ontology position: ∅ is taken as a nullary operation and “operational ontology” is renamed the operational universe/foundation; no mathematics is altered. This revision was carried out with Claude Opus 4.8.